

ThreatGraph

AI-Powered Threat Intelligence & Investigation Platform

Design Document -- 3-Day Build Specification
Version 1.0 | September 2026

ThreatGraph is a production-style AI investigation platform that transforms submitted digital indicators into structured, searchable, and connected intelligence. This document defines the architecture, data model, AI integration, and build plan for a focused 3-day MVP that demonstrates a complete, production-oriented engineering stack.

Layer	Technology
API framework	FastAPI + Python 3.12
Async processing	Redis Streams + asyncio workers
Database	PostgreSQL 16 with tsvector full-text search
AI integration	Anthropic Claude with structured tool calling
Deployment	Docker Compose
Build target	3 days -- single vertical slice, production practices

Contents

1. Project Overview
2. MVP Scope -- What Was Cut and Why
3. Technology Stack
4. System Architecture
5. Data Model
6. Investigation Lifecycle
7. Enrichment Pipeline
8. AI Analyst
9. Event-Driven Processing (Redis Streams)
10. API Reference
11. 3-Day Build Plan
12. Testing Strategy

1. Project Overview

ThreatGraph is an AI-powered threat intelligence and investigation platform. It accepts digital indicators -- domains, URLs, email addresses, and social profiles -- and builds a structured, searchable, connected view of the infrastructure associated with those indicators.

The project is scoped as a focused engineering demonstration rather than a complete enterprise security product. Its purpose is to show how an AI reasoning layer can operate reliably inside an observable, production-oriented backend system.

Core Design Principle

AI is used for reasoning and interpretation -- not as the source of truth. Deterministic enrichment services collect the underlying evidence. The AI analyst receives that structured evidence and uses it to explain findings, identify relationships, and produce a grounded conclusion. Every AI claim must reference a specific enrichment observation stored in PostgreSQL.

Engineering Disciplines Demonstrated

- Distributed backend development with FastAPI and asyncio
- Asynchronous event-driven processing with Redis Streams consumer groups
- Relational data modeling with PostgreSQL -- entities, relationships, observations
- AI agent orchestration with structured tool calling and schema-validated output
- **Evidence grounding:** AI conclusions linked to specific observation records in the database
- Reliability engineering: idempotency, retry, and partial failure classification
- Containerised deployment with Docker Compose
- Structured logging with investigation trace IDs propagated across all services

2. MVP Scope -- What Was Cut and Why

The original design specification describes a 3-6 month engineering effort. For a 3-day portfolio build, the scope was reduced by removing infrastructure complexity while preserving every architectural feature that demonstrates engineering depth.

What Was Cut

Component	Reason for removal
Kubernetes + Terraform + AWS	Docker Compose describes the same architecture at zero setup cost. K8s-readiness is demonstrated through stateless service design, not by running a local cluster.
Elasticsearch	Replaced with PostgreSQL full-text search (tsvector + pg_trgm). Reduces operational complexity without sacrificing the search engineering demonstration. The architecture remains Elasticsearch-ready as an optional future read layer.
Prometheus + Grafana + OpenTelemetry	Replaced with structured JSON logging, investigation_id trace fields on every log line, and a /metrics endpoint in Prometheus text format.
Next.js frontend	FastAPI's Swagger UI fully demonstrates the API. The backend is the artifact.
Email and social profile investigations	Domain and URL investigations provide one complete, polished vertical slice. Additional indicator types are documented as planned extensions.

What Was Kept

These components are non-negotiable -- they carry the signal that makes ThreatGraph valuable:

- **AI analyst with structured tool calling** -- the defining feature of the platform.
- **Evidence grounding** -- AI conclusions linked to specific observation UUIDs, validated before persistence.
- **Redis Streams** -- real event-driven architecture with consumer groups, XACK, and XPENDING-based retry.
- **PostgreSQL relationship model** -- entities, relationships, and investigations with explicit relationship types.
- **Investigation lifecycle state machine** -- pending, enriching, analyzing, correlating, complete.
- **Idempotent workers** -- safe to restart, resistant to duplicate message delivery.
- **Structured logging with trace IDs** -- a single investigation ID traceable across all services.

The vertical slice principle: one indicator enters the platform, evidence is collected and persisted, background workers process the investigation, and an AI analyst produces an evidence-grounded structured result. This single flow, executed flawlessly, demonstrates more engineering depth than a partial implementation of a sprawling stack.

3. Technology Stack

Layer	Technology	Role
API	FastAPI	REST API, request validation, async route handlers
Async processing	Redis Streams	XADD/XREADGROUP, consumer groups, XPENDING retry
Database	PostgreSQL 16	Durable state, relationship model, full-text search
Search	pg_trgm + tsvector	Built-in PostgreSQL FTS -- no separate search cluster
AI	Anthropic Claude	Structured tool calling, schema-validated output
Workers	Python asyncio	Enrichment, analysis, and correlation workers
Containers	Docker Compose	API, workers, Redis, PostgreSQL in one file
CI/CD	GitHub Actions	Lint, test, and build on push

Key Technology Decisions

PostgreSQL instead of Elasticsearch. Full-text search with tsvector and pg_trgm handles indicator lookups and structured filtering without a separate cluster. The architecture remains Elasticsearch-ready if throughput demands grow later.

Docker Compose instead of Kubernetes. The application is designed stateless (API and workers) with stateful components (PostgreSQL, Redis) treated as persistent infrastructure. This is the same separation Kubernetes enforces, described at zero overhead for a portfolio project.

Redis Streams instead of Kafka or RabbitMQ. Redis Streams provides consumer groups, message acknowledgment, and pending-entry inspection with minimal operational complexity -- the right tool for this scale.

4. System Architecture

ThreatGraph uses a service-oriented backend organised around five responsibilities: ingestion, enrichment, analysis, correlation, and persistence. Each is handled by a dedicated component with a clearly defined scope.

Components

Component	Responsibility
FastAPI (API service)	Accepts investigation submissions, serves status and results, exposes search. Publishes tasks to Redis. Stateless -- can scale horizontally.
Redis Streams	Asynchronous task queue. Decouples the API from workers. Provides durable delivery, consumer groups, and at-least-once delivery semantics.
Enrichment worker	Runs DNS and HTTP enrichment in parallel per investigation. Normalises results into observations and writes them to PostgreSQL. Re-publishes a type: analyze task on completion.
AI analyst worker	Receives structured evidence. Calls Claude with a constrained 5-tool set. Validates the structured response with Pydantic. Writes the result to PostgreSQL.
Correlation worker	Reads the AI result and enrichment observations. Creates entity and relationship records in PostgreSQL. Marks the investigation complete.
PostgreSQL	Single source of truth for all durable state: investigations, observations, entities, relationships, and results.

Request Flow

- 1 Client sends POST /investigations with an indicator, e.g. suspicious-domain.com.
- 2 FastAPI validates the request, saves an investigation record with status = pending, and immediately returns the investigation ID. No blocking on enrichment.
- 3 FastAPI publishes a type: enrich task to the Redis tasks stream.
- 4 The enrichment worker picks up the task, sets status = enriching, and runs DNS and HTTP enrichment in parallel. Each result is saved as an observation record in PostgreSQL.
- 5 On completion, the enrichment worker publishes a type: analyze task. The AI analyst retrieves all observations, calls Claude with structured tools, and writes the validated result.
- 6 The correlation worker creates entity and relationship records, then marks the investigation complete.
- 7 The client polls GET /investigations/{id} and receives the complete investigation with result, supporting evidence, and all discovered relationships.

5. Data Model

All application state lives in PostgreSQL. The schema uses five tables whose relationships allow the system to move from a single indicator to a full graph of connected infrastructure.

investigations

Primary record for each submitted indicator. Tracks the investigation lifecycle.

Column	Description
id	uuid PK. Also used as a trace ID in all logs and worker messages.
indicator	The submitted value, e.g. suspicious-domain.com.
indicator_type	One of: domain, url. (email, social defined as future extensions.)
status	Current stage: pending, enriching, analyzing, correlating, complete, failed, cancelled.
created_at / updated_at	Creation time and last status-change timestamp.

entities

Deduplicated registry of all observed indicators and infrastructure. Shared across investigations.

Column	Description
id	uuid PK.
type	Entity class: domain, ip, url, certificate.
value	text UNIQUE. Enables upsert when the same IP appears across multiple investigations.
metadata	jsonb. Flexible extra data (ASN, registrar, etc.) without schema changes.
first_seen / last_seen	Tracks when this entity was first and most recently observed.

observations

Every piece of enrichment evidence collected during an investigation. The evidence layer that AI conclusions must reference.

Column	Description
id	uuid PK. Referenced by investigation_results.evidence_refs to ground AI conclusions.
investigation_id	uuid FK to investigations.

Column	Description
entity_id	uuid FK to entities. The entity this evidence describes.
source	Which enrichment service produced this: dns, http, tls, or whois.
data	jsonb. Normalised evidence payload -- DNS records, HTTP headers, certificate data.
collected_at	When the evidence was collected.

relationships

Durable records of connections between entities. Enables infrastructure correlation across investigations.

Column	Description
id	uuid PK.
source_entity_id	uuid FK to entities. The entity the relationship originates from.
target_entity_id	uuid FK to entities. The entity the relationship points to.
relationship_type	Explicit type: resolves_to, uses_certificate, redirects_to, or hosted_on.
confidence	float 0.0-1.0. Confidence score assigned during correlation.
observed_at	When this relationship was first recorded.

investigation_results

The AI analyst's validated output. One result per investigation.

Column	Description
id	uuid PK.
investigation_id	uuid FK UNIQUE. One-to-one with investigations.
classification	One of: benign, suspicious, malicious, unknown.
confidence	float 0.0-1.0. Confidence in the classification.
explanation	AI-generated narrative. Must reference observations from evidence_refs.
entity_ids	jsonb array. UUIDs of entities relevant to this conclusion.
evidence_refs	jsonb array. Observation UUIDs supporting this conclusion. Min 1 required.
recommended_action	One of: monitor, block, escalate, no_action, request_review.

Column	Description
created_at	When the result was persisted after Pydantic validation.

Evidence grounding: investigation_results.evidence_refs holds a JSON array of observation UUIDs. Every AI conclusion must cite at least one supporting observation. This is enforced by Pydantic validation before any result touches PostgreSQL, making the AI analyst's reasoning transparent and auditable.

6. Investigation Lifecycle

Every investigation progresses through a defined set of statuses stored in PostgreSQL. Status transitions are durable, making the system resilient to worker restarts -- a worker that picks up an investigation mid-flight can check the current status and resume from the correct stage.

Status Transitions

Status	Set by	Trigger	Output
pending	API	POST /investigations received	Investigation ID issued to client
enriching	Enrichment worker	type: enrich task consumed	DNS + HTTP observations written to PostgreSQL
analyzing	AI analyst	type: analyze task consumed	Claude called; result validated and persisted
correlating	Correlation worker	type: correlate task consumed	Entity and relationship records created
complete	Correlation worker	All stages successful	Result available via GET /investigations/{id}
failed	Any worker	Unrecoverable error after max retries	Error logged; investigation remains queryable
cancelled	API	DELETE /investigations/{id}	Workers skip on cancelled flag

Partial Failure Handling

A failed enrichment step does not invalidate the investigation. Three failure modes are distinguished:

- **Transient failures** (network timeout, DNS error): retried up to 3 times with exponential backoff.
- **Incomplete enrichment** (one source unavailable): the investigation continues with available evidence. The AI analyst is informed which sources are missing.
- **Permanent failures** (invalid indicator format, schema validation error): investigation marked failed immediately without retrying.

7. Enrichment Pipeline

Enrichment converts a raw indicator into structured, normalised evidence. The MVP focuses on domain and URL investigation with two deterministic sources that require no external API keys.

Evidence Sources

Source	Data Collected	Library	Signal Value
DNS	A, AAAA, MX, NS, TXT, CNAME records	dnspython	Resolved IPs, nameservers, SPF/DMARC policy
HTTP	Status code, headers, redirect chain, server fingerprint, page title	aiohttp	Technology stack, hosting provider, redirect behaviour

Parallel Execution

DNS and HTTP enrichment run concurrently using `asyncio.gather` with `return_exceptions=True`. If one source fails the other continues. Results are written as independent observation records so partial evidence is preserved and queryable.

```
dns_result, http_result = await asyncio.gather(
    enrich_dns(indicator),
    enrich_http(indicator),
    return_exceptions=True # partial failure is acceptable
)

if not isinstance(dns_result, Exception):
    await save_observation(db, investigation_id, "dns", dns_result)
if not isinstance(http_result, Exception):
    await save_observation(db, investigation_id, "http", http_result)
```

Future Enrichment Sources

The observation table's JSONB data column means new sources can be added without schema changes. Planned extensions: TLS certificate inspection (SANs, issuer, validity period), WHOIS data (registrar, registration date), and content-based signals (embedded links, page text patterns).

8. AI Analyst

The AI analyst is the reasoning layer of ThreatGraph. It does not collect data -- it interprets data collected by the deterministic pipeline. The analyst receives structured enrichment evidence and uses a constrained set of tools to investigate relationships, inspect observations, and produce a validated, evidence-grounded conclusion.

Design Principles

Evidence grounding. Every AI claim must be linked to a specific observation UUID. Validated by Pydantic before persistence. Conclusions without supporting evidence are rejected.

Constrained tool set. The agent has access to exactly 5 tools. A smaller set produces cleaner decision-making, easier auditing, and more predictable behaviour.

No fact invention. The system prompt explicitly prohibits stating facts not present in enrichment data. Missing information leads to a low-confidence unknown classification, not a fabricated answer.

Schema-validated output. The `create_investigation_result` tool enforces a strict Pydantic schema before any result touches PostgreSQL. Invalid responses are rejected and logged.

Single result call. The agent is instructed to call `create_investigation_result` exactly once, after gathering sufficient evidence. This prevents duplicate or partial results.

Tool Set

Tool	Purpose	When to Call
<code>search_indicators</code>	Search existing entities and past investigations by value or keyword.	First -- detect previously seen infrastructure.
<code>get_enrichment</code>	Retrieve all enrichment observations for a specific entity.	Before drawing any conclusion about an entity.
<code>get_related_entities</code>	Traverse the relationship graph. Returns connected IPs, certificates, and domains.	To detect shared infrastructure across investigations.
<code>inspect_observation</code>	Retrieve full raw data from a specific observation (e.g. complete certificate SANs).	When summary data is insufficient for a conclusion.
<code>create_investigation_result</code>	Write the final validated result. Requires classification, confidence, explanation, <code>entity_ids</code> , <code>evidence_refs</code> (min 1), and <code>recommended_action</code> .	Exactly once, after sufficient evidence is gathered.

Structured Output Schema

```
class InvestigationResult(BaseModel):
    classification: Literal[
        "benign", "suspicious", "malicious", "unknown"
    ]
    confidence: float = Field(ge=0.0, le=1.0)
    explanation: str = Field(min_length=20, max_length=1000)
    entity_ids: list[str] = Field(min_length=1)
    evidence_refs: list[str] = Field(min_length=1) # observation UUIDs
    recommended_action: Literal[
        "monitor", "block", "escalate",
        "no_action", "request_review"
    ]
```

Typical Tool Call Sequence

A standard investigation uses 3-5 tool calls before producing a result:

1. `search_indicators` -- check whether the domain has been seen in past investigations.
2. `get_enrichment` -- retrieve DNS and HTTP observations for the primary entity.
3. `get_related_entities` -- check whether resolved IPs or certificates appear elsewhere.
4. `inspect_observation` (optional) -- retrieve full certificate SAN list or raw header data.
5. `create_investigation_result` -- write the validated conclusion with evidence references.

9. Event-Driven Processing (Redis Streams)

Redis Streams provides the asynchronous processing layer. The API publishes tasks and workers consume them independently. Workers do not communicate directly with each other -- they communicate by writing results to PostgreSQL and publishing new tasks to the stream.

Stream and Consumer Group Structure

Name	Type	Role
tasks	Stream	Single stream. All task types flow through one stream.
enrichment-cg	Consumer group	Reads type: enrich messages. DNS + HTTP workers.
analyst-cg	Consumer group	Reads type: analyze messages. AI analyst worker.
correlation-cg	Consumer group	Reads type: correlate messages. Correlation worker.

Pipeline Chaining

Workers chain the pipeline by re-publishing to the stream after completing their stage. When the enrichment worker finishes, it publishes a type: analyze message. When the AI analyst finishes, it publishes type: correlate. Neither the API nor any worker needs to know what stage comes next -- the task type carries that information.

Retry Mechanism

Redis Streams' XPENDING command lists messages that have been delivered but not acknowledged (XACK). This is the basis for detecting and recovering from failed workers:

1. Worker calls XREADGROUP to receive a task message.
2. Worker processes the task and calls XACK on success.
3. If the worker crashes mid-processing, no XACK is sent. The message stays pending.
4. The retry loop calls XPENDING to find messages unacknowledged for over 30 seconds.
5. XCLAIM reassigns the stuck message to another available worker in the same consumer group.
6. After 3 failed attempts, the task is re-published as type: dlq_retry for manual inspection.

Idempotency

Workers use investigation IDs as idempotency keys. Before writing an observation or relationship record, the worker checks whether that record already exists. Re-delivery of a task therefore produces no duplicate records. Status updates use conditional PostgreSQL UPDATES (WHERE status = current_stage) to prevent backwards transitions.

10. API Reference

The API exposes eight endpoints across three concerns: investigation management, intelligence search, and operations. All responses use typed Pydantic models with consistent error formats.

Endpoints

Method	Path	Description
POST	/investigations	Submit an indicator. Returns investigation_id immediately. Body: { indicator, indicator_type }. Processing is async.
GET	/investigations/{id}	Status, result, and metadata. Poll until status = complete or failed. Includes AI result when ready.
GET	/investigations/{id}/evidence	All enrichment observations. Optional: ?source=dns http tls whois. Returns the raw evidence behind the AI conclusion.
GET	/investigations/{id}/entities	Related entities discovered during enrichment. Optional: ?relationship_type=resolves_to uses_certificate ...
GET	/search	Full-text + structured search. Params: q (required), type, status, classification, limit, offset.
GET	/health	Liveness check. Pings PostgreSQL and Redis. Returns per-dependency status. Used by Docker Compose healthcheck.
GET	/metrics	Prometheus text format. Exposes investigation counts by status, queue depth, and worker latency counters.
DELETE	/investigations/{id}	Cancel a pending investigation. Workers check for the cancelled flag before processing each stage.

Error Handling

All endpoints return consistent JSON error responses: { "detail": "message", "code": "ERROR_CODE" }. Validation errors return 422 with field-level detail. Not found returns 404. All 500 responses include the investigation_id trace field for log correlation.

11. 3-Day Build Plan

The build is organised as three sequential days, each ending with a testable milestone. Day 3 ends with a complete, demo-ready investigation flow.

Day 1 -- Foundation

Goal: The entire backend skeleton runs in Docker Compose. An investigation can be submitted and its status retrieved. No enrichment or AI processing yet.

- PostgreSQL schema: all 5 tables, indexes, tsvector FTS column on investigations
- FastAPI application: all routes, Pydantic models, dependency injection for DB and Redis
- POST /investigations and GET /investigations/{id} working end-to-end
- Redis Streams plumbing: XADD on submission, XREADGROUP boilerplate for each worker
- Investigation lifecycle state machine: conditional status transitions in PostgreSQL
- Docker Compose: API, PostgreSQL, Redis services with health checks
- Structured JSON logging with investigation_id as a trace field on every log line

Day 2 -- Intelligence

Goal: A submitted domain is fully enriched and analysed by Claude. The complete investigation result is queryable with evidence references intact.

- DNS enrichment: A, AAAA, MX, NS, TXT records via dnspython
- HTTP enrichment: status code, headers, redirect chain via aiohttp
- Parallel enrichment with asyncio.gather and partial failure handling
- Observation records written to PostgreSQL with normalised JSONB payloads
- AI analyst worker: Claude API with all 5 tools wired to real database queries
- Structured output validation with Pydantic before persistence
- Correlation worker: entity upsert, relationship creation, investigation marked complete
- GET /investigations/{id}/evidence and /entities routes
- Retry logic: XPENDING + XCLAIM in each worker loop

Day 3 -- Polish and Demonstration

Goal: The project is demo-ready. A reviewer can clone the repo, run docker-compose up, submit an investigation, and see a grounded result with supporting evidence.

- Search endpoint: PostgreSQL FTS with tsvector + pg_trgm, structured filtering
- /health and /metrics endpoints (Prometheus text format)
- End-to-end test: submit a domain, assert complete status, validate result schema, verify evidence_refs point to real observation records
- At least one AI evaluation fixture: a known-evidence set that produces a consistent classification
- README with architecture overview, Docker setup instructions, and a sample investigation walkthrough
- Code cleanup: consistent error handling, type hints throughout, no debug prints

Definition of done: A reviewer runs docker-compose up, submits POST /investigations with a domain, polls GET /investigations/{id} until status = complete, and receives an AI-generated classification with evidence_refs pointing to real enrichment observations. That flow, working reliably, is the entire deliverable.

12. Testing Strategy

Testing covers the system at four levels. The AI component has deterministic evaluation fixtures in addition to standard integration tests, ensuring the agent produces valid, grounded output and does not invent unsupported facts.

Unit tests

Scope: Individual functions with no I/O.

- DNS record parsing and HTTP header normalisation functions
- Pydantic schema validation: valid and invalid investigation result structures
- Status transition logic: valid transitions pass, invalid ones raise an error
- Relationship deduplication: same entity pair + type does not create a duplicate record

Integration tests

Scope: Service interactions against real (test) databases.

- PostgreSQL: write and retrieve observations; FTS search returns expected results
- Redis: XADD task, XREADGROUP pickup, XACK, XPENDING detection after missed ACK
- Worker task routing: type: enrich task is consumed only by the enrichment consumer group

End-to-end test

Scope: Full investigation flow against a running Docker Compose stack.

- Submit POST /investigations with a controlled test domain
- Poll GET /investigations/{id} until status = complete (timeout: 60 seconds)
- Assert classification is one of the four valid enum values
- Assert evidence_refs is non-empty and all UUIDs resolve to real observation records
- Assert GET /investigations/{id}/evidence returns records matching evidence_refs

AI evaluation fixtures

Scope: Deterministic AI behaviour validation.

- Known-malicious evidence set: assert classification = malicious, confidence ≥ 0.7
- Known-benign evidence set: assert classification $\neq$ malicious
- Incomplete evidence fixture: assert classification = unknown, confidence < 0.5
- All fixtures: assert evidence_refs items exist in the fixture observation set -- no invention